

Generics

Wildcards

why?

when?

how?



returns a sorted
list of integers

takes an unsorted
list of integers

```
public class SortingClass {  
    public List<Integer> sort(List<Integer> unsortedList) {  
        // does the sorting here  
        // return sortedList  
    }  
}
```



```
public class SortingClass {  
  
    public List<String> sort(List<String> unsortedList) {  
        // does the sorting here  
        // return sortedList  
    }  
}
```

1

```
public class SortingClass {  
  
    public List<????> sort(List<????> unsortedList) {  
        // does the sorting here  
        // return sortedList  
    }  
}
```

2



we've been using
generics all this time



the List class is a perfect example of Java generics

```
List<Integer>  
List<String>  
List<Double>  
List<MyObject>  
.  
.  
.
```

using the **same**
class: "List"



```
public class SortingClass<T> {  
    public List<Integer> sort(List<Integer> unsortedList) {  
        // does the sorting here  
        // return sortedList  
    }  
}
```



```
SortingClass<String> sortingObj = new SortingClass<>();
```

```
public class SortingClass<T> {  
    public List<T> sort(List<T> unsortedList) {  
        // does the sorting here  
        // return sortedList  
    }  
}
```



T = type | **K** = key | **V** = value

```
public class SortingClass<like> {  
  
    public List<like> sort(List<like> unsortedList) {  
        // does the sorting here  
        // return sortedList  
    }  
  
}
```

```
public class SortingClass<T, K, V> {  
  
    public List<T> sort(List<T> unsortedList) {  
        // does the sorting here  
        // return sortedList  
    }  
  
}
```



can't we just modify
the method?



What is a Wildcard?

represented by a question mark (?)

it stands for an unknown type

use it to make a single method generic
without affecting the whole class



```
public class SortingClass {  
    public List<?> sort(List<?> unsortedList) {  
        // does the sorting here  
        // return sortedList  
    }  
}
```



```
List list = Arrays.asList(3, 2, 1, 5, 4);
```

```
List list = Arrays.asList("B", "A", "E", "D", "C");
```

```
public class SortingClass {  
  
    public List<?> sort(List<?> unsortedList) {  
        // does the sorting here  
        // return sortedList  
    }  
  
}
```

```
List list = Arrays.asList(2.0, "A", 5, "D", 3);
```

this will throw a runtime exception



```
List<Integer> list = asList(3, 2, 1, 5, 4);
```

```
List<String> list = asList("B", "A", "E", "D", "C");
```

```
public class SortingClass {  
    public List<?> sort(List<?> unsortedList) {  
        // does the sorting here  
        // return sortedList  
    }  
}
```

```
List<String> list = asList(2.0, "A", 5, "D", 3);
```

this will throw a compile error



Best Practice

always specify the list's type

avoid raw types like just List

always write `List<String>`, `List<Integer>`, etc.



```
public class SortingClass {
```

```
    public List<?> sort(List<? extends Number> unsortedList) {
```

```
        // does the sorting here
```

```
        // return sortedList
```

```
    }
```

```
}
```



bounded wildcards

```
public class SortingClass {
```

```
    public List<? extends Number> sort(List<? extends Number> unsortedList) {
```

```
        // does the sorting here
```

```
        // return sortedList
```

```
    }
```

```
}
```

**help mitigate all
runtime exceptions to
compilation errors**



```
public class SortingClass<T extends Number & Comparable<T>>> {  
    public List<T> sort(List<T> unsortedList) {  
        // does the sorting here  
        // return sortedList  
    }  
}
```

multiple bounds can be
used on both generics
and wildcards



why can't we use **primitive**
types with **generics**?

```
List<int>
```



```
List<Integer>
```



when compiling the **generic**
parameter T will be **erased** and
replaced by the “**Object**” class
“**int**” does not extend “**Object**”
but “**Integer**” does



plain generic

unbounded wildcards

upper bounded wildcards

lower bounded wildcards



```
public class MainApp {
```

```
    public static void doArrayStuff(Object[] objects) {  
        // do stuff with the array  
    }
```

```
}
```



```
public class MainApp {  
  
    public static void main(String[] args) {  
        String[] strings = new String[10];  
        doArrayStuff(strings);  
    }  
  
    public static void doArrayStuff(Object[] objects) {  
        // do stuff with the array  
    }  
  
}
```



```
public class MainApp {  
  
    public static void main(String[] args) {  
        List<String> stringList = new ArrayList<>();  
        doListStuff(stringList);  
    }  
  
    public static void doListStuff(List<Object> objects) {  
        // do stuff with the list  
    }  
  
}
```





```
public class MainApp {  
  
    public static void main(String[] args) {  
        List<String> stringList = new ArrayList<>();  
        doListStuff(stringList);  
    }  
  
    public static void doListStuff(List<Object> objects) {  
        // do stuff with the list  
    }  
}
```

Required type: List <Object>
Provided: List <String>

Collection types and Generics
in Java are **Invariant**



```
public class MainApp {  
  
    public static void main(String[] args) {  
        String[] strings = new String[10];  
        doArrayStuff(strings);  
    }  
  
    public static void doArrayStuff(Object[] objects) {  
        // do stuff with the array  
        objects[0] = 10;  
    }  
  
    throws a RuntimeException!  
}
```

**invariance shifted our errors
from runtime to compile time**



Covariance

if we have a type `T`, only type `T` and subtypes of `T` are allowed in this context

Contravariance

`<? extends T>`

Invariance

arrays in Java
are covariant



Covariance

if we have a type T , only type T and supertypes of T are allowed in this context

Contravariance



$\langle ? \text{ super } T \rangle$

Invariance



Covariance

$\langle T \rangle$

Contravariance

Invariance

if we have a type T , only type T
is allowed in this context



```
public static void main(String[] args) {  
    List<Integer> integers = Arrays.asList(1, 2, 3);  
    double sum = sum(integers);  
}
```

Required type: Collection <Number>

Provided: List <Integer>

```
public static double sum(Collection<Number> numbers) {  
    return numbers.stream()  
        .mapToDouble(Number::doubleValue)  
        .sum();  
}
```



```
public static void main(String[] args) {  
    List<Integer> integers = Arrays.asList(1, 2, 3);  
    double sum = sum(integers); // 6  
}
```

```
public static double sum(Collection<? extends Number> numbers) {  
    return numbers.stream()  
        .mapToDouble(Number::doubleValue)  
        .sum();  
}
```



```
public static void main(String[] args) {  
    List<Integer> integers = Arrays.asList(1, 2, 3);  
    add(integers);  
}
```

```
public static void add(Collection<? extends Number> numbers) {  
    numbers.add(10); // Compile Error  
    numbers.add(null); // Passes  
}
```



`<? extends T>`

used when we need to retrieve the
elements of a collection and not when we
need to alter the extended collection



```
public static void main(String[] args) {  
    List<Number> numbers = new ArrayList<>();  
    append(numbers);  
    numbers.add(3.14);  
}
```

Required type:	Collection	<Integer>
Provided:	List	<Number>

```
public static void append(Collection<? extends Integer> integers) {  
    IntStream.rangeClosed(1, 5).forEach(integers::add);  
}
```



Why “extends” Doesn’t Work Here

Number is a supertype of Integer, not a subtype

the extends-wildcard only allows T and its subtypes

Number doesn’t qualify as a subtype of Integer



```
public static void main(String[] args) {  
    List<Number> numbers = new ArrayList<>();  
    append(numbers);  
    numbers.add(3.14); // 1, 2, 3, 4, 5, 3.14  
}
```



```
public static void append(Collection<? super Integer> integers) {  
    IntStream.rangeClosed(1, 5).forEach(integers::add);  
}
```



```
public static void main(String[] args) {  
    List<Number> numbers = new ArrayList<>();  
    append(numbers);  
    numbers.add(3.14); // 1, 2, 3, 4, 5, 3.14  
  
    List<Object> objects = new ArrayList<>();  
    append(objects);  
    objects.add("Like"); // 1, 2, 3, 4, 5, Like  
}
```

sum of this list can't be retrieved

```
public static void append(Collection<? super Integer> integers) {  
    IntStream.rangeClosed(1, 5).forEach(integers::add);  
}
```



Why “super” Can’t Retrieve

the compiler can’t guarantee a consistent type

a list with super can store integers, strings, and other types

the compiler cannot execute the sum
operation on mixed types



<? super T>

used when we need to alter the
collection and not when we need to
retrieve the elements of the collection



PECS

Producer Extends Consumer Super

when we are passing in a parameter from which we are producing Ts, we want to get Ts out of this parameter, then we should use the type `<? extends T>`

when we are passing in a parameter into which we want to put Ts, the thing consumes elements of type T, then we should use the type `<? super T>`



PECS

Producer Extends Consumer Super

use extends to get values out of a structure

use super to put values into a structure

don't use a wildcard to both get and put



PECS

Producer Extends Consumer Super

| | | Bounded Parameter Produces Ts? | |
|--------------------------------|-----|--|--|
| Bounded Parameter Consumes Ts? | | Yes | No |
| | Yes | <code>MyClass<T></code>
(Invariant in T) | <code>MyClass<? super T></code>
(Contravariant in T) |
| | No | <code>MyClass<? extends T></code>
(Covariant in T) | <code>MyClass<?></code>
(Independent of T) |



Wildcards with Return Types

our APIs should. just. work.

clients making use of our APIs shouldn't have to
think about passing specific parameter types

will strip the clients of the flexibility we
guaranteed as we made it more difficult to use

